Intune Compliance-Failure Diagnostics & Remediation

Implementation Solution Guide

Version 1.0.0 · 2026-07-01

Contents

Executive Summary 4

The problem . . . 4

The solution . . . 4

What is included . . . 4

Business value . . . 4

Bottom line . . . 5

Solution Overview 5

What this solution does . . . 5

How compliance evaluation actually works . . . 5

Evaluation timing — why results lag . . . 6

Where results surface (know every window) . . . 6

The diagnostic model . . . 6

Requirements 7

R1. Role to read compliance policies and device state . . . 7

R2. Role to read Conditional Access and sign-in logs . . . 7

R3. Microsoft Graph read access for the scripts . . . 8

R4. PowerShell 7 with the Microsoft Graph SDK installed . . . 8

R5. Access to at least one affected (non-compliant) device . . . 8

R6. A pilot / target security group for staged remediation . . . 9

Requirements summary . . . 9

Global Requirements — Intune Solutions (category-level) 9

G1. Microsoft Entra ID tenant . . . 10

G2. Microsoft Intune subscription . . . 10

G3. Administrative roles . . . 10

G4. Device enrollment in Intune . . . 10

G5. Pilot security group . . . 10

G6. Supported Windows baseline . . . 10

G7. Network access to Microsoft cloud endpoints . . . 10

Diagnostic Playbook 11

Step 0 — Frame the symptom . . . 11

Step 1 — Which setting failed? (read per-setting state) . . . 11

Step 2 — Is it timing, or a real failure? . . . 12

Step 3A — Built-in setting failed . . . 12

Step 3B — Custom compliance rule, or an "Error" state . . . 13

Step 4 — Has the grace period expired? . . . 13

Step 5 — What does Conditional Access do about it? (blast radius) . . . 14
Step 6 — Fleet pattern (many devices, same setting) . . . 14
Step 7 — Remediate safely (never fleet-wide-disable) . . . 15
Testing & Validation 15
Gate 1 — The failing setting passes . . . 15
Gate 2 — The device is compliant . . . 16
Gate 3 — Conditional Access allows the user . . . 16
Regression / safety checks . . . 17
Overall pass criteria . . . 17
Troubleshooting 17
Best Practices 19
1. Diagnose to the setting, always . . . 19
2. Use grace deliberately, not as a hiding place . . . 19
3. Enforce in stages: Assess → Configure → Pilot → Validate → Expand . . . 19
4. Never disable or loosen policy fleet-wide to unblock one device . . . 20
5. Understand the three clocks . . . 20
6. Keep built-in and Custom compliance rules distinct . . . 20
7. Map the Conditional Access blast radius before you act . . . 20
8. Keep the toolkit read-only by default . . . 20
9. Document the resolution . . . 20
Deliverables Summary 20
Documents . . . 21
PowerShell scripts . . . 21
How to use them together . . . 22
Outcome . . . 22
Appendix A — PowerShell Scripts 23

Executive Summary

The problem

A managed device flips to **non-compliant**. Conditional Access immediately blocks the user from email, files, and apps. The help desk cannot say **which setting failed**, **why**, or **when access will return** — so the fastest-looking fix becomes the most dangerous one: disabling or loosening the compliance policy, which strips protection from the entire fleet to unblock one person.

The solution

Intune Compliance-Failure Diagnostics & Remediation replaces guesswork with a **repeatable root-cause method** and a set of read-only-by-default PowerShell scripts. It teaches exactly how Intune decides compliance — **policy** → **per-setting state** → **grace period** → **device state** → **Conditional Access gate** — and gives a symptom-driven decision tree to read the **per-setting failure reason**, tell **evaluation lag from a real failure**, check **grace expiry**, distinguish **built-in from Custom compliance** rules, and measure the **Conditional Access blast radius**. Then it remediates the **cause** — safely, staged to a pilot group — **without ever disabling policy fleet-wide**.

What is included

- **Diagnostic Playbook** — the symptom tree, with exact Intune/Entra navigation and a script check at every branch.
- **Five PowerShell scripts** — compliance-state discovery, per-device per-setting failure validation, a fleet compliance + Conditional Access-impact report, safe remediation of common failures, and a pilot/target-group builder. Discovery, Validation, and Report are read-only; Remediation and Set-ScopeTargeting default to **-WhatIf** (preview first).
- **Requirements, Testing, Troubleshooting, and Best Practices** — including the three pass/fail gates (setting passes → device compliant → CA allows) and grace/staged- enforcement guidance.

Business value

- **Faster resolution** — every non-compliant device explained to the exact failing setting, cutting help-desk time and user downtime.
- **No self-inflicted exposure** — the method fixes the cause without weakening compliance for everyone; safe to run mid-incident because the tools read by default.
- **Clear impact** — you can state the Conditional Access blast radius (who is blocked, by which policy) before you act.
- **A capability you keep** — a documented method and reusable scripts, at a fraction of the cost of engaging a consultant for each incident.

Bottom line

Know exactly **why** a device is non-compliant — the failing setting, the timing and grace, and the Conditional Access impact — and remediate it **safely**, using a method and scripts your team keeps and reuses.

Solution Overview

What this solution does

A device turns up **non-compliant** in Intune. Conditional Access blocks the user. The help desk cannot say *which* setting failed, *why* it failed, or *when* it will clear. People start guessing — and the worst guess is "disable the compliance policy so the user can work," which strips protection from the whole fleet.

This solution replaces the guesswork with a **root-cause method**. It teaches you exactly how Intune decides a device is compliant or not, where each result surfaces, and gives you a symptom-driven decision tree plus five PowerShell scripts to read the **per-setting failure reason**, understand the **evaluation timing and grace period**, see the **Conditional Access blast radius**, and remediate **safely** — without turning policy off for everyone.

How compliance evaluation actually works

Compliance in Intune is a chain. A break anywhere in the chain shows up at the end as "non-compliant," so you must read the chain from left to right:

Policy → **per-setting state** → **grace period** → **device compliance state** → **Conditional Access gate**

1. **Compliance policy.** In the admin center you author a **Compliance policy** under

Devices → **Manage devices** → **Compliance** (older tenants: **Devices** → **Compliance policies** → **Policies**). A policy is a named object (for example `Win-Baseline-Compliance`) that contains one or more **settings** — BitLocker, Secure Boot, Minimum OS version, Firewall, Antivirus, Defender signatures, a **Custom compliance** script result, and so on. The policy is **assigned** to a group.

1. **Per-setting state.** The device's Intune management extension evaluates **each

setting independently **and reports a state per setting:** Compliant**,** Not compliant**,** Error**, or** Not applicable**. This is the single most important fact in the whole solution:** a device is only as compliant as its worst setting.** One failing setting makes the whole device non-compliant, even if every other setting passes. The per-setting result — including the failure reason string — is what your diagnosis is built on.

1. **Grace period (Actions for noncompliance).** Each compliance policy has an **Actions

for noncompliance **tab. The default action is** Mark device noncompliant **with a schedule of** Immediately (0 days)**, but you can set a grace period (for example** `1 day` **or** `3 days`**). During grace, the device has *failed a setting* but is still reported** Compliant **to Conditional Access — the clock is running. When the grace period** expires**, the device flips to** Not compliant **and CA can block it. A device that "was fine yesterday and is blocked today" with no config change is almost always a** grace expiry**, not a new failure.

1. **Device compliance state.** Intune rolls the per-setting states (and grace) up into

one **device compliance state** shown on the device blade and in reports. If the device has policies from more than one source, the **least-compliant** result wins.

1. **Conditional Access gate.** Conditional Access lives in **Entra** (not Intune) under

Protection → **Conditional Access** → **Policies**. A CA policy with the grant control **Require device to be marked as compliant** reads the device compliance state at sign-in. Compliant → access granted. Not compliant → access blocked (or a different grant applied). **CA is the thing the user feels** — but the *cause* is upstream in the per-setting state. Never diagnose the block at the CA layer alone.

Evaluation timing — why results lag

Compliance is **not** instant. Understanding the timing prevents the most common false alarm ("I fixed it but it still says non-compliant"):

- The device evaluates compliance on a **check-in cycle** (typically every ~8 hours, and

shortly after enrollment/restart). A fix applied on the device is not reflected until the **next check-in and report**.

- Forcing a sync (device **Settings** → **Accounts** → **Access work or school** → **Info** → **Sync**, or

Intune **device** → **Sync**) triggers an earlier check-in, but the **report** in the console still needs a few minutes to update.

- Conditional Access reads the **last reported** state. Even after the device reports

compliant, the user may need to **sign out and back in** (or wait for token refresh) for CA to re-read it.

So there are three clocks to keep straight: **device evaluation**, **report freshness**, and **CA token/sign-in**. A "still failing" symptom is often just one of these clocks not having ticked yet.

Where results surface (know every window)

You will read the same failure from several places; each shows a different slice:

- **Per device:** Intune → **Devices** → **All devices** → **<device>** → **Device compliance**. Shows

each assigned policy and, expanding a policy, the **per-setting** state and the **failure reason**. This is your primary diagnostic surface.

- **Per policy:** Intune → **Devices** → **Compliance** → **<policy>** → **Device status** /

Per-setting status. Shows how many devices fail, and which setting fails most.

- **Fleet:** Intune → **Devices** → **Monitor** → **Noncompliant devices**, and **Reports →

Device compliance**. Shows the population and trend.

- **Conditional Access impact:** Entra → **Protection** → **Conditional Access** → **Policies**

(which policies require compliance and who they target), and **Sign-in logs** (filter by user/device to see whether a CA "Require compliant device" control **failed** at sign-in). This is how you measure the **blast radius** of a non-compliant device.

The diagnostic model

Every diagnosis in this solution answers five questions, in order:

1. **Which setting failed?** (read the per-setting state and reason — never assume)
2. **Is this timing, or a real failure?** (evaluation lag vs an actual failing setting)
3. **Has grace expired?** (failed-but-in-grace vs failed-and-enforced)
4. **Built-in or custom rule?** (a native setting vs a Custom compliance script result)
5. **What does Conditional Access do about it?** (the blast radius — who is blocked)

Answer those five and you know the root cause and the safe fix. The **Diagnostic Playbook** (`03-implementation.md`) is the symptom tree that walks each branch; the scripts read the raw data so you are never guessing.

Requirements

The category-level prerequisites (Entra tenant, Intune subscription, enrollment, Windows baseline, network access) are covered in **global-requirements-intune-solutions.md**. Confirm those first. Below are the prerequisites **specific to diagnosing compliance failures** — the roles, Graph read access, and device access you need to read per-setting state, evaluation timing, and Conditional Access impact, and to remediate safely.

Each item is written as **Description / Why / Verify / Implement if missing.**

R1. Role to read compliance policies and device state

Description: An Intune role that can **read** compliance policies, device compliance state, and per-setting status across the fleet. **Intune Administrator** covers everything; for a read-only operator, the built-in **Help Desk Operator** or a custom role with **Device compliance policies: Read** and **Organization: Read** is enough for diagnosis.

Why: The whole method is reading per-setting failure reasons, grace configuration, and device compliance state. Without read on compliance objects you cannot see *which* setting failed.

Verify: Sign in to `https://intune.microsoft.com`. Open **Devices** → **Compliance** and confirm the policy list loads; open any device → **Device compliance** and confirm you can expand a policy to per-setting state.

Implement if missing: Entra → **Roles and administrators** → assign **Intune Administrator** (or **Help Desk Operator** for read-only). For least privilege, Intune → **Tenant administration** → **Roles** → **Create** a custom role granting **Device compliance policies: Read** and assign it with a scope group.

R2. Role to read Conditional Access and sign-in logs

Description: An Entra role that can **read** Conditional Access policies and **Sign-in logs**. **Security Reader** grants both read-only; **Conditional Access Administrator** is needed only if you will edit CA.

Why: The Conditional Access blast radius — *who* is blocked by a non-compliant device and *which* CA policy did it — is read from CA policy definitions and the sign-in logs. This is how you scope impact before

touching anything.

Verify: Entra → **Protection** → **Conditional Access** → **Policies** lists policies; Entra → **Monitoring & health** → **Sign-in logs** loads and can be filtered by user.

Implement if missing: Entra → **Roles and administrators** → assign **Security Reader** to the operator (add **Conditional Access Administrator** only if remediation will edit CA).

R3. Microsoft Graph read access for the scripts

Description: The five PowerShell scripts use the **Microsoft Graph PowerShell SDK** and sign in interactively with delegated permissions. The read/report scripts need **DeviceManagementConfiguration.Read.All**, **DeviceManagementManagedDevices.Read.All**, and **Policy.Read.All** (for Conditional Access). The remediation and scope-targeting scripts additionally need the matching **.ReadWrite.All** scopes, but default to **-WhatIf** so they only *read* until you explicitly commit.

Why: The scripts read per-setting compliance state, device state, and CA policies directly from Graph so your diagnosis does not depend on hand-reading the console. The first interactive sign-in raises an admin **consent** prompt for these scopes.

Verify: Run `Connect-MgGraph -Scopes "DeviceManagementManagedDevices.Read.All"` and confirm sign-in succeeds; `Get-MgContext` lists the scope. Then `Get-MgDeviceManagementManagedDevice -Top 1` returns a device without an authorization error.

Implement if missing: Install the SDK with `Install-Module Microsoft.Graph -Scope CurrentUser`. On first `Connect-MgGraph`, an admin must **Accept** the consent prompt (or a Global Administrator can pre-consent in Entra → **Identity** → **Applications** → **Enterprise applications** → **Microsoft Graph Command Line Tools** → **Permissions** → **Grant admin consent**).

R4. PowerShell 7 with the Microsoft Graph SDK installed

Description: PowerShell 7.x (recommended) or Windows PowerShell 5.1 with the **Microsoft.Graph** module available to the account running the scripts.

Why: The scripts are written against the Graph SDK cmdlets and paginate large result sets; a current PowerShell avoids TLS and module-loading issues.

Verify: `$PSVersionTable.PSVersion` shows 7.x (or 5.1); `Get-Module Microsoft.Graph -ListAvailable` returns the module.

Implement if missing: Install PowerShell 7 from the Microsoft Store or `https://aka.ms/powershell`, then `Install-Module Microsoft.Graph -Scope CurrentUser -Force`.

R5. Access to at least one affected (non-compliant) device

Description: Local or remote (Intune remote-help / RDP) access to a device that is currently reporting a compliance failure, plus local admin on it.

Why: Some failures (a real BitLocker-off, a stopped firewall, a Custom compliance script returning a fail) are confirmed and fixed **on the device**, not in the console. You also confirm the fix by forcing a sync and

watching the state clear.

Verify: You can sign in to the affected device and open **Settings** → **Accounts** → **Access work or school** → **<account>** → **Info**, which shows the managed-device sync status and a **Sync** button.

Implement if missing: Arrange remote access (Intune **Remote Help**, RDP, or physical), and ensure the operator has local administrator rights to inspect BitLocker, firewall, Defender, and Custom compliance script output.

R6. A pilot / target security group for staged remediation

Description: A Microsoft Entra security group (see global **G5**) used to pilot any compliance-policy change or grace adjustment before it reaches the fleet.

Why: The cardinal rule of this solution is **never disable or loosen a compliance policy fleet-wide** to fix one device. Staged changes go to a pilot group first; the `Set-ScopeTargeting` script builds and validates that targeting and defaults to **-WhatIf**.

Verify: Entra → **Groups**; the pilot group (for example `Compliance-Diag-Pilot`) exists and has the intended members.

Implement if missing: Entra → **Groups** → **New group** → **Security**, name it, add the affected/test devices or users.

Requirements summary

#	Requirement	Read-only?	Needed for
R1	Intune role: read compliance + device state	Yes	Diagnosis (per-setting state)
R2	Entra role: read CA + sign-in logs	Yes	Conditional Access blast radius
R3	Graph read scopes (Read.All)	Yes	All scripts (report/discovery/validation)
R4	PowerShell 7 + Microsoft.Graph SDK	n/a	Running the scripts
R5	Access to an affected device (local admin)	No	Confirming/fixing on-device failures
R6	Pilot / target group	n/a	Safe staged remediation

Global Requirements — Intune Solutions (category-level)

These requirements apply to **every** solution in the **Intune Solutions** category. They are published once at the category level so individual listings stay small. Confirm these before deploying *any* Intune solution. Each entry: **Description · Why · Verify · Implement-if-missing.**

G1. Microsoft Entra ID tenant

Description: An active Microsoft Entra ID (Azure AD) tenant. **Why:** It is the identity backbone for every cloud and Intune-managed resource. **Verify:** Sign in to `https://entra.microsoft.com` as an administrator. **Implement:** Create a Microsoft 365/Azure subscription, which provisions a tenant.

G2. Microsoft Intune subscription

Description: An Intune-bearing license plan. **Why:** Intune delivers all configuration, compliance, and security policies. **Verify:** `https://intune.microsoft.com` loads; **Tenant administration** → **Tenant status** shows active. **Implement:** Assign Microsoft 365 E3/E5, Business Premium, or standalone Intune Plan 1.

G3. Administrative roles

Description: Appropriate Entra/Intune admin roles for the operator. **Why:** Creating policies and enrollment settings requires elevated rights. **Verify:** Entra → **Roles and administrators**; confirm **Intune Administrator**. **Implement:** Assign **Intune Administrator**; reserve Global Administrator for tenant-wide changes.

G4. Device enrollment in Intune

Description: Targeted Windows devices are enrolled and manageable. **Why:** Policies only reach enrolled devices. **Verify:** Intune → **Devices** → **All devices**; the device appears and is **compliant**. **Implement:** Enable **Automatic Enrollment** and join devices to Entra ID.

G5. Pilot security group

Description: A Microsoft Entra security group for pilot/targeted users or devices. **Why:** Every solution is piloted before fleet rollout. **Verify:** Entra → **Groups**; the pilot group exists. **Implement:** Entra → **Groups** → **New group** → **Security** → add members.

G6. Supported Windows baseline

Description: Windows 10 21H2+ or Windows 11 with current updates. **Why:** Modern Intune features assume a patched, supported OS. **Verify:** `winver`, or Intune → device → **Hardware**. **Implement:** Deploy **Update rings** or upgrade to Windows 11.

G7. Network access to Microsoft cloud endpoints

Description: Outbound HTTPS to Microsoft 365, Entra, and Intune service URLs. **Why:** Enrollment, policy sync, and authentication depend on these endpoints. **Verify:** From a device, reach `https://login.microsoftonline.com` and `https://enrollment.manage.microsoft.com`. **Implement:** Allow Microsoft's published Office 365/Intune URL & IP ranges through proxy/firewall.

The standard deployment model (all solutions): Assess → Configure → Pilot → Validate → Expand.

Diagnostic Playbook

This is the symptom-driven decision tree for a compliance failure. Work it **top to bottom** — each step narrows the cause before you touch anything. The golden rule runs through every branch: **read the exact per-setting failure first, understand timing and grace, measure the Conditional Access blast radius, then remediate the smallest thing. Never disable or loosen a compliance policy fleet-wide to unblock one device.**

Run the scripts alongside the clicks — the console tells you *what*, the scripts give you the raw reason string and the fleet/CA context so you are never guessing.

Step 0 — Frame the symptom

Before diagnosing, write down what the user actually experiences. Pick the entry symptom:

- **A. A specific device is non-compliant** → start at Step 1.
- **B. "It worked yesterday, blocked today," no change made** → likely **grace expiry**;

jump to Step 4 after confirming the setting in Step 1–2.

- **C. Many devices failed the same setting at once** → likely a **policy/setting change or

a service-side shift**; do Step 1–3 on one device, then use the fleet view (Step 6).

- **D. User is blocked but device shows compliant** → this is a **Conditional Access /

timing** problem, not a setting failure; jump to Step 5.

Step 1 — Which setting failed? (read per-setting state)

Never trust "non-compliant" alone. Find the exact failing setting.

1. Intune → **Devices** → **All devices** → **<device>**.
2. Open **Device compliance**. Each assigned compliance policy is listed with its state.
3. **Expand the failing policy.** You now see the **per-setting** state: each setting shows

Compliant / Not compliant / Error / Not applicable, plus a **failure reason** on the ones that failed.

1. Record the exact policy name (for example `Win-Baseline-Compliance`), the exact failing

setting name, its **state**, and the **reason string** verbatim.

Run in parallel: `INTCOMPDX-Validation.ps1 -DeviceName "<device>"` returns the same per-device, per-setting failure with the raw reason from Graph — copy that string; it is the authoritative text.

Branch on the state:

- State **Not compliant** with a reason → a real setting failure → **Step 2**.
- State **Error** → the setting could not be evaluated (agent/script/permission problem),

not necessarily a real failure → **Step 3B**.

• State **Not applicable** on a setting you expected to enforce → targeting/OS mismatch → **Step 3A**.

• No failing setting visible but device still non-compliant → **timing** → **Step 5**.

Step 2 — Is it timing, or a real failure?

A device can *report* a failure that is already fixed but not yet re-evaluated.

1. Note the **Last check-in** / **Last compliance report** time on the device blade.

2. If the reported failure is older than the last on-device change, force a re-evaluation:

device **Settings** → **Accounts** → **Access work or school** → **<account>** → **Info** → **Sync**, or Intune **device** → **…** → **Sync**.

1. Wait for the next check-in (minutes, not instant — see Overview timing), then re-read

Step 1.

• Failure **clears** after sync → it was **stale reporting**; done, confirm CA in Step 5.

• Failure **persists** after a fresh check-in → it is a **real setting failure** →

continue.

Now split by rule type:

• The failing setting is a **built-in** setting (BitLocker, Secure Boot, Minimum OS

version, Firewall, Antivirus, Defender signatures, password/PIN) → **Step 3A**.

• The failing setting is a **Custom compliance** result (a policy that references a

Custom compliance script + JSON) → **Step 3B**.

Step 3A — Built-in setting failed

Confirm the real state **on the device**, because a built-in setting reflects an actual OS condition:

• **BitLocker / Encryption:** on the device run `manage-bde -status` (or check **Settings →

Privacy & security → Device encryption**). If the OS drive is** not **fully encrypted / protection is** off**, the failure is genuine — the fix is to encrypt, not to remove the setting.

• **Secure Boot / TPM:** `msinfo32` → **Secure Boot State = On**, **TPM** present and ready.

Hardware/firmware condition — fix in UEFI/enable TPM.

• **Minimum OS version:** the device build is below the policy's floor. Fix = **patch/update

the device** (Update rings), not lower the floor.

• **Firewall / Antivirus / Defender signatures:** confirm the service is running and

signatures are current (`Get-MpComputerStatus`). Restart the service / update signatures.

• **Password / PIN complexity:** the device's local policy does not meet the rule; fix the

device-side setting.

Read the exact policy floor and setting definition with `INTCOMPDX-Discovery.ps1 -PolicyName "<policy>"` so you compare the device's real value against the **exact** configured requirement.

Now check grace before deciding urgency → **Step 4**. Safe fixes for the common built-in failures are in `INTCOMPDX-Remediation.ps1` (default **-WhatIf**).

Step 3B — Custom compliance rule, or an "Error" state

Custom compliance = a compliance policy that runs a **Custom compliance script** (PowerShell) whose output is matched against a **JSON rule file** (the discovered settings, operators, and remediation strings you authored). It fails differently from built-ins:

1. In the failing policy, confirm it is a **Custom compliance** policy and note the

referenced **script** name and the **JSON** rule that failed (the per-setting reason shows the rule's *SettingName* and often your authored **RemediationString**).

1. On the device, run the same **detection** the script performs to see the real returned

value; compare it to the **operator + value** in the JSON rule.

1. Decide which is wrong:

• The **device value** genuinely fails the rule → real failure, fix the device.

• The **rule/JSON** is wrong (bad operator, wrong data type, typo in *SettingName*) →

the *policy* is faulty; fix the JSON/script and re-upload (stage via **Step 7**, do **not** delete the policy fleet-wide).

• State is **Error** (not "Not compliant") → the script did not return valid JSON, timed

out, or the agent could not run it. This is an **evaluation** failure, not a compliance verdict: check the script's exit/output on the device, script signing, and the Intune management extension health. Fix the script/agent; the setting cannot be "compliant" until it evaluates cleanly.

`INTCOMPDX-Validation.ps1` flags Custom vs built-in and surfaces the Error state so you do not mistake an evaluation error for a real failure.

Step 4 — Has the grace period expired?

Grace decides whether a *failed* setting is actually *enforced* yet.

1. Intune → **Devices** → **Compliance** → **<policy>** → **Properties** → **Actions for noncompliance**.
2. Read the **Mark device noncompliant** schedule: **Immediately (0 days)** = no grace;

`N days` = a grace window.

1. Compare **when the setting first failed** (from the device's compliance history) against

the grace window:

• Still **within grace** → the device is failing a setting but is **reported compliant**

to CA; the user is **not** blocked yet. You have `N days` to fix the real setting (Step 3). Do **not** extend grace to hide it.

• Grace **expired** → the device is now **Not compliant** and CA will act. Fix the real

setting (Step 3) and let it re-evaluate; only consider a *temporary, pilot-scoped* grace extension if the fix legitimately needs more time (Step 7).

Symptom B ("worked yesterday, blocked today," no change) is almost always a **grace expiry of a setting that had been failing quietly** — Step 1 will show the setting, Step 4 confirms the clock ran out.

`INTCOMPDX-Discovery.ps1` reports each policy's grace configuration so you can see at a glance which policies enforce immediately versus with a window.

Step 5 — What does Conditional Access do about it? (blast radius)

Now measure impact — *who* is blocked and *by which* policy.

1. Entra → **Protection** → **Conditional Access** → **Policies**. Identify every **enabled** policy

with the grant control **Require device to be marked as compliant** (or a device-filter requiring compliance). Note each policy's **Users/groups**, **Target resources** (cloud apps), and **conditions**.

1. Determine whether the affected user/device is **in scope** of one of those policies — if

not, a non-compliant device may not actually be blocked (the block is coming from somewhere else).

1. Entra → **Monitoring & health** → **Sign-in logs** → filter by the user. Open a failed

sign-in → **Conditional Access** tab → find the policy whose result is **Failure** and confirm the failing control is **Require compliant device**. This proves CA is the block and names the exact policy.

1. If the device **reports compliant** but CA still blocks → it is **timing**: CA read a

stale state or the user's token predates the fix. Have the user **sign out and back in** (or wait for token refresh) after the device reports compliant.

`INTCOMPDX-Report.ps1` produces the fleet compliance summary **plus** the CA-impact view (which compliance-requiring CA policies exist and how many affected devices they touch), so you can state the blast radius precisely before remediating.

Step 6 — Fleet pattern (many devices, same setting)

If the failure is widespread (Symptom C):

1. Intune → **Devices** → **Compliance** → **<policy>** → **Per-setting status** to see which single

setting is failing across many devices.

1. Ask what changed: a **policy edit** (new/tightened setting), a **service-side** shift

(updated Defender platform, new OS floor), or a **broken Custom compliance script/JSON** pushed to everyone.

1. Fix the **cause**, staged to the pilot group first (Step 7) — do not mass-exempt devices

or disable the policy.

`INTCOMPDX-Report.ps1` ranks settings by failure count so you spot the one setting driving the fleet number.

Step 7 — Remediate safely (never fleet-wide-disable)

Choose the **smallest** action that fixes the **cause**:

- **Real device failure** (BitLocker off, firewall stopped, out-of-date signatures, low OS

build): fix on the device / push the enabling policy; let it re-evaluate. `INTCOMPDX-Remediation.ps1` performs the safe common fixes and defaults to **-WhatIf** — review the planned action, then re-run with `-WhatIf:$false` to commit.

- **Faulty rule/JSON or script** (Custom compliance): correct the rule/script and stage the

updated policy to the **pilot group** first.

- **Grace genuinely too short** for a legitimate fleet-wide fix: adjust grace on a

pilot-scoped assignment, validate, then expand — never blanket-extend to mask a real failure.

- **Targeting change** (a device should not have been in scope): use

`INTCOMPDX-Set-ScopeTargeting.ps1` to move the affected device(s) to the correct pilot/ target group. It defaults to **-WhatIf** and shows the assignment change before applying.

After any change, **confirm the loop closes** (Step 5): device re-evaluates → reports compliant → CA re-reads → user regains access. Re-run the report to prove the fleet number moved in the right direction. Verification gates are in `05-testing.md`.

Testing & Validation

A compliance diagnosis is only "done" when three gates all pass, in order: **the failing setting now passes** → **the device is reported compliant** → **Conditional Access allows the user.** Test each gate explicitly; do not stop at the first green.

Run the checks from the same surfaces you diagnosed from, and confirm with the scripts so the result is not just a stale console.

Gate 1 — The failing setting passes

Goal: the specific setting that failed now reports **Compliant** (not just the device overall).

Steps:

1. Force a re-evaluation on the device (Sync), then wait for the next check-in and report.
2. Intune → **Devices** → **All devices** → **<device>** → **Device compliance** → expand the policy →

confirm the previously failing **setting** now shows **Compliant** (state, not just absence of an error).

1. Script check: `INTCOMPDX-Validation.ps1 -DeviceName "<device>"` returns **no** failing

settings for that device (empty failure set for the policy).

Result	Meaning
PASS	The named setting reports **Compliant**; validation returns no failures.
FAIL	The setting still shows **Not compliant** or **Error** → return to Diagnostic Playbook Step 3 (real failure) or Step 3B (evaluation error).

Gate 2 — The device is compliant

Goal: the **device-level** compliance state is **Compliant**, and grace is not masking a still-failing setting.

Steps:

1. Intune → **Devices** → **All devices** → **<device>** → the device state shows **Compliant**.
2. Confirm this is a *real* pass, not a *grace* pass: re-check **Actions for noncompliance**

(Playbook Step 4) — the device should be compliant because the setting passed, not because it is still inside a grace window.

1. Script check: `INTCOMPDX-Discovery.ps1 -DeviceName "<device>"` shows the device

Compliant with no settings in a failing/grace state.

Result	Meaning
PASS	Device state is **Compliant** on its own merits (no failing setting hiding under grace).
FAIL	Device still **Not compliant**, or compliant only because grace has not expired → the underlying setting is not fixed; return to Step 3.

Gate 3 — Conditional Access allows the user

Goal: the user can actually sign in to the protected resource — the block is gone.

Steps:

1. Have the user **sign out and sign back in** (or wait for token refresh) so CA re-reads

the fresh compliant state.

1. Entra → **Monitoring & health** → **Sign-in logs** → filter by the user → open the latest

sign-in → **Conditional Access** tab → the **Require compliant device** control now shows **Success** for the relevant policy.

1. Confirm the user reaches the resource that was previously blocked.
2. Script check: `INTCOMPDX-Report.ps1` shows the device no longer counted against the

compliance-requiring CA policies (blast radius reduced by one).

Result	Meaning
PASS	Sign-in log shows the compliance control **Success**; user reaches the resource.
FAIL	CA control still **Failure** → if the device is compliant, it is **timing** (stale token/read): re-sign-in and re-test; if it persists, re-check which CA policy is in scope (Playbook Step 5).

Regression / safety checks

Because the cardinal rule is **do not loosen policy fleet-wide**, verify you did not widen exposure while fixing one device:

• **No fleet-wide disable:** the compliance policy is still **assigned and enabled** for its

original groups (you fixed the cause, not the policy). `INTCOMPDX-Discovery.ps1` should show the policy unchanged in scope.

• **Staged change stayed staged:** any grace/targeting change went only to the **pilot

group** first. `INTCOMPDX-Set-ScopeTargeting.ps1 -WhatIf` re-run shows the intended assignment and no unexpected broader targeting.

• **Fleet trend moved the right way:** re-run `INTCOMPDX-Report.ps1` — the non-compliant

count for the fixed setting decreased; no *new* setting started failing as a side effect.

Overall pass criteria

The diagnosis is complete when **all three gates PASS and the regression checks hold**: the exact setting passes, the device is genuinely compliant, Conditional Access grants access, and no policy was disabled or loosened for the fleet to get there.

Troubleshooting

Common problems seen **while running this diagnostic solution itself** — reading state, running the scripts, and confirming a fix. For diagnosing the *device's* compliance failure, use the **Diagnostic Playbook** (`03-implementation.md`); this page fixes the toolkit and the console around it. Each entry: **Symptom → Cause → Fix.**

The console shows non-compliant but no setting looks failed

Symptom: Device state is **Not compliant**, but expanding the policy shows every setting Compliant (or blank). **Cause:** Report lag — the device evaluated a fix but the per-setting view has not refreshed; or a second policy from another assignment is the one failing. **Fix:** Force a **Sync**, wait for the next check-in and report, re-open the blade. Check **all** assigned policies on the device, not just the obvious one — the

least-compliant policy wins. Confirm with `INTCOMPDX-Validation.ps1 -DeviceName "<device>"`.

`Connect-MgGraph` *fails or scopes are missing*

Symptom: Sign-in errors, or `Get-MgContext` does not list the needed scope; cmdlets return **authorization** / 403 errors. **Cause:** The Microsoft Graph SDK is not installed, admin consent was never granted, or you connected without the required scopes. **Fix:** `Install-Module Microsoft.Graph -Scope CurrentUser`, then `Connect-MgGraph -Scopes "DeviceManagementConfiguration.Read.All","DeviceManagementManagedDevices.Read.All","Policy.Read.All"` and **Accept** the consent prompt (or have a Global Administrator grant admin consent to **Microsoft Graph Command Line Tools**). Re-check with `Get-MgContext`.

Scripts return no devices / empty results

Symptom: Discovery/Report run cleanly but return nothing. **Cause:** Missing read scope, a scope/RBAC boundary limiting what the account can see, or a filter (device name / policy name) that does not match. **Fix:** Confirm scopes (above) and that the account's Intune role can read compliance and devices (R1). Check the exact object name in the console and reuse it verbatim — names are case- and spelling-sensitive. Run without a name filter first to confirm data returns.

A setting shows "Error" instead of Not compliant

Symptom: Per-setting state is **Error**. **Cause:** The setting could not be **evaluated** — a Custom compliance script did not return valid JSON, timed out, was blocked by script signing, or the Intune management extension is unhealthy. This is **not** a compliance verdict. **Fix:** On the device, run the detection script manually and confirm it emits valid JSON; check script signing and the management extension. Fix the script/agent — the setting cannot report Compliant until it evaluates cleanly. See Playbook Step 3B.

Fixed the device but Conditional Access still blocks the user

Symptom: Device reports **Compliant** but the user is still blocked. **Cause:** Timing across the three clocks — CA read a stale state, or the user's existing token predates the fix. **Fix:** Confirm device state is Compliant, then have the user **sign out and back in** (or wait for token refresh). Re-check the **Sign-in logs** → **Conditional Access** tab for the compliance control turning **Success**. See Playbook Step 5.

Grace makes a failing device look compliant

Symptom: A device with a genuinely failing setting reports **Compliant** and is not blocked. **Cause:** The setting is failing but still **within the grace window** (Actions for noncompliance), so CA is not enforcing yet. **Fix:** This is expected behaviour, not a bug — but do not ignore it. Fix the real setting before grace expires. Do **not** extend grace to keep it hidden. See Playbook Step 4.

Remediation script "did nothing"

Symptom: `INTCOMPDX-Remediation.ps1` (or `Set-ScopeTargeting`) reported planned actions but changed nothing. **Cause:** Both default to **-WhatIf** — they only preview. **Fix:** Review the previewed actions, then re-run with `-WhatIf:$false` to commit. This is intentional so diagnosis is always safe to run.

Widespread failure appeared suddenly across many devices

Symptom: Many devices flipped non-compliant on the same setting at once. **Cause:** A policy edit, a broken Custom compliance script/JSON pushed to everyone, or a service-side change (new Defender platform, OS floor). **Fix:** Use `INTCOMPDX-Report.ps1` to confirm the single setting driving the count, find what changed, fix the **cause** staged to a pilot group — do not mass-exempt or disable the policy. See Playbook Step 6.

"Not applicable" on a setting you expected to enforce

Symptom: A setting shows **Not applicable** rather than Compliant/Not compliant. **Cause:** The setting does not apply to that device's OS/edition, or the policy is not truly assigned to that device. **Fix:** Confirm the policy assignment reaches the device and that the setting applies to its OS/edition. Use `INTCOMPDX-Discovery.ps1` to verify policy targeting.

If a symptom is not listed here, return to the **Diagnostic Playbook** and work the tree from Step 0 — the failure is almost always one of: timing, grace, a real setting, a custom-rule error, or Conditional Access.

Best Practices

These practices keep compliance both **enforced** and **diagnosable** — so failures are rare, explained fast, and never "fixed" by weakening protection for everyone.

1. Diagnose to the setting, always

Never act on "non-compliant" alone. The first move is always to read the **per-setting state and the exact reason string**. The device is only as compliant as its worst setting; if you do not know which setting failed, you cannot fix the right thing — and you risk disabling a whole policy to chase one setting.

2. Use grace deliberately, not as a hiding place

- Set a **grace period** (Actions for noncompliance) that matches how long a legitimate fix

realistically takes (for example `1–3 days` for BitLocker to finish encrypting), so a transient/in-progress state does not instantly block a user.

- **Never extend grace to mask a real, persistent failure.** Grace buys time to fix the

cause; it is not a remediation. A setting failing quietly inside grace is a ticking clock — fix it before it expires (this is the classic "worked yesterday, blocked today" story).

- Keep grace **consistent** within a policy so the evaluation timing is predictable.

3. Enforce in stages: Assess → Configure → Pilot → Validate → Expand

- Roll out new or tightened compliance settings to a **pilot group first**, watch the

per-setting status, then expand. A setting that fails 500 devices at once is almost always a rollout that skipped piloting.

- Stage grace changes and Custom compliance script/JSON updates the same way.

4. Never disable or loosen policy fleet-wide to unblock one device

This is the cardinal rule. Fix the **cause** (the device, the rule, or the targeting), not the policy. To unblock a single device fast without stripping protection:

- Fix the real setting on the device and force a sync; or
- Move the device to a correctly-scoped pilot/target group; or
- Apply a *temporary, pilot-scoped* exception — never a global disable.

5. Understand the three clocks

Keep **device evaluation**, **report freshness**, and **CA token/sign-in** separate in your head. Most "I fixed it but it's still blocked" reports are one clock that has not ticked: force a sync, wait for the report, then have the user re-sign-in so CA re-reads the state.

6. Keep built-in and Custom compliance rules distinct

- Prefer **built-in settings** where they exist — they evaluate reliably and report clear

reasons.

- For **Custom compliance**, treat the **script + JSON** as code: validate the script

returns valid JSON, use correct operators/data types, and author a helpful **RemediationString** so the per-setting reason tells the help desk what to do. An **Error** state means the rule could not evaluate — that is a script/agent bug, not a compliance verdict.

7. Map the Conditional Access blast radius before you act

Know which enabled CA policies **Require device to be marked as compliant**, and who they target, *before* a device goes non-compliant. Then an incident is "this device is blocked from these resources for these users," not a surprise. Review CA scope and compliance policies together, periodically.

8. Keep the toolkit read-only by default

Run Discovery, Validation, and Report freely — they only read. Keep Remediation and Set-ScopeTargeting on their **-WhatIf** default; review the planned change, then commit explicitly. This makes diagnosis safe to run any time, including during an active incident.

9. Document the resolution

For each incident record: the **failing setting + reason**, whether it was **timing / grace / real failure / custom-rule / CA**, the **fix applied**, and the **blast radius**. Patterns across incidents tell you which setting to harden at the source (Best Practice 3) so the same failure stops recurring.

Deliverables Summary

Everything included in **Intune Compliance-Failure Diagnostics & Remediation**, and how the pieces fit together.

Documents

Document	What it gives you
Solution Overview (`01`)	How compliance evaluation works end-to-end (policy → per-setting state → grace → device state → Conditional Access gate), the three clocks of evaluation timing, where every result surfaces, and the five-question diagnostic model.
Requirements Guide (`02`)	Each prerequisite — Intune roles, Entra/CA read roles, Graph read scopes, PowerShell + SDK, affected-device access, pilot group — with **Description / Why / Verify / Implement if missing**.
Global Requirements	Category-level prerequisites shared by all Intune solutions (tenant, subscription, enrollment, Windows baseline, network).
Diagnostic Playbook (`03`)	The symptom-driven decision tree: which setting failed, timing vs real failure, grace expiry, built-in vs Custom rule, and the Conditional Access blast radius — with exact clicks and script checks per branch.
Testing & Validation (`05`)	The three pass/fail gates — setting passes, device compliant, CA allows — plus regression checks that no policy was loosened fleet-wide.
Troubleshooting (`06`)	Fixes for problems running the toolkit itself: Graph consent, empty results, Error states, CA timing, grace masking, WhatIf previews.
Best Practices (`07`)	Grace discipline, staged enforcement, never-disable-fleet-wide, the three clocks, built-in vs Custom rules, mapping CA blast radius, read-only-by-default tooling.

PowerShell scripts

All scripts use the **Microsoft Graph PowerShell SDK**. Discovery, Validation, and Report are **read-only**. Remediation and Set-ScopeTargeting default to **-WhatIf** (preview only) and change nothing until you re-run with `-WhatIf:$false`.

Script	Read/Write	Purpose
`INTCOMPDX-Discovery.ps1`	Read-only	Enumerate compliance policies, their settings, grace (Actions for noncompliance), assignments, and device compliance state.
`INTCOMPDX-Validation.ps1`	Read-only	For a device (or set), return the **per-setting** failure with the raw reason string, and flag built-in vs Custom vs Error states.
`INTCOMPDX-Report.ps1`	Read-only	Fleet compliance summary ranked by failing setting, **plus** the Conditional Access impact (which compliance-requiring CA policies exist and how many affected devices they touch).
`INTCOMPDX-Remediation.ps1`	Write (**-WhatIf** default)	Apply safe, common fixes (sync, re-target, staged grace) with a previewed plan before committing.
`INTCOMPDX-Set-ScopeTargeting.ps1`	Write (**-WhatIf** default)	Build/adjust pilot/target group assignments for staged remediation, showing the assignment change before applying.

How to use them together

1. **Discovery** — map the policies, settings, grace, and where a device stands.
2. **Validation** — read the exact per-setting failure and reason on the affected device.
3. Work the **Diagnostic Playbook** to classify the cause (timing / grace / real / custom / CA).
1. **Report** — measure the Conditional Access blast radius and fleet pattern.
2. **Remediation** / **Set-ScopeTargeting** (preview, then commit) — fix the cause safely, staged to a pilot group; never disable policy fleet-wide.
1. **Testing** — confirm all three gates pass and no policy was loosened.

Outcome

Every non-compliant device explained to the **exact failing setting**, with its **evaluation timing and grace status** and its **Conditional Access impact**, and a **safe remediation** — using a repeatable method and scripts you keep and reuse.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

INTCOMPDX-Discovery.ps1

Purpose: Builds the compliance map for diagnosis: every compliance policy, the grace configuration (Actions for noncompliance), assignments, and where devices stand. Optionally scope to one policy or one device by name. Read-only against Microsoft Graph. Replace the sample values such as your-policy-name / your-device-name. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\INTCOMPDX-Discovery.ps1

How to run: powershell -ExecutionPolicy Bypass -File .\INTCOMPDX-Discovery.ps1

INTCOMPDX-Remediation.ps1

Purpose: Applies the safe, common remediations for a compliance failure: force a device re-evaluation (sync) so a fixed setting re-reports, and preview a staged grace adjustment on a named policy. Defaults to -WhatIf so it only previews; re-run with -WhatIf:$false to commit. Never disables or loosens a policy fleet-wide. Replace the sample values your-device-name / your-policy-name. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\INTCOMPDX-Remediation.ps1 -DeviceName your-device-name (preview only; add -WhatIf:$false to commit)

How to run: powershell -ExecutionPolicy Bypass -File .\INTCOMPDX-Remediation.ps1 -DeviceName your-device-name (preview only; add -WhatIf:$false to commit)

INTCOMPDX-Report.ps1

Purpose: Produces the fleet picture for triage: how many devices are compliant vs not, which settings fail most, and the Conditional Access blast radius — which enabled CA policies require a compliant device and how many affected devices they touch. Read-only. Optionally export to CSV; replace the sample path your-report-path.csv. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\INTCOMPDX-Report.ps1

How to run: powershell -ExecutionPolicy Bypass -File .\INTCOMPDX-Report.ps1

INTCOMPDX-Set-ScopeTargeting.ps1

Purpose: Moves affected device(s) or user(s) into the correct pilot/target Entra security group so a compliance fix or grace change can be staged before the fleet. Defaults to -WhatIf so it only previews the membership change; re-run with -WhatIf:$false to commit. Never disables a policy or widens fleet targeting. Replace the sample values your-group-name / your-object-id. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\INTCOMPDX-Set-ScopeTargeting.ps1 -GroupName your-group-name -MemberObjectId your-object-id (preview only; add -WhatIf:$false to commit)

How to run: powershell -ExecutionPolicy Bypass -File .\INTCOMPDX-Set-ScopeTargeting.ps1 -GroupName your-group-name -MemberObjectId your-object-id (preview only; add -WhatIf:$false to commit)

INTCOMPDX-Validation.ps1

Purpose: For a named device (or all non-compliant devices), returns each failing setting with its state and the raw failure reason string, and flags whether the setting is built-in, a Custom compliance rule, or an

Error (could-not-evaluate) state. This is the authoritative per-setting failure for diagnosis. Read-only. Replace the sample value your-device-name with a real device name. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\INTCOMPDX-Validation.ps1 -DeviceName your-device-name

How to run: powershell -ExecutionPolicy Bypass -File .\INTCOMPDX-Validation.ps1 -DeviceName your-device-name